FACTORY PRODUCTION DEPLOYMENT REPORT

SmartShield IPS

OT/ICS Edition

Version 1.1.16 · Factory Build

Protocols detected: Modbus · S7Comm · DNP3 · PROFINET · CIP · NTP/PTP

Features: OT-safe blocking · single-connection volumetric flood detection · Redis persistence · web auth · SIEM syslog export

Built on Stratosphere Linux IPS (SLIPS) · GPL-2.0

Zeek 8 · Redis 8.x · Python 3.10+ · Ubuntu 22.04 / 24.04 LTS

Contents

1. System Overview

SmartShield is a Python/Zeek-based Intrusion Prevention System originally built as a graduation project running inside Docker on a laptop. The factory edition transforms it into a production-ready IPS capable of protecting Operational Technology (OT) and Industrial Control System (ICS) environments from real attacks.

The system sits passively on a Linux server at the IT/OT network boundary, receiving a mirrored copy of all traffic via a SPAN port or passive TAP. It detects **19 categories of OT/ICS attacks** across six industrial protocols, generates structured evidence, and optionally blocks attackers through iptables — with a critical OT-safe mechanism that prevents any PLC, HMI, or SCADA server from ever being auto-blocked.

What's new in v1.1.16 THIS RELEASE

- **Single-connection volumetric flood detection** — catches floods that ride one long-lived TCP/UDP connection (e.g. `hping3 --flood`), which Zeek records as a single flow and which earlier per-connection counters missed entirely.
- **Dedicated S7 authorized-master list** (`config/s7_authorized_masters.conf`) — S7Comm unauthorized read/write checks now use their own list instead of borrowing the Modbus one; falls back to the Modbus list automatically if the S7 file is absent.
- **Updated base platform** — SLIPS 1.1.16 core: Zeek 8 support, Redis security patch (CVE-2025-49844), bloom-filter speedups in evidence/whitelist handling, and faster evidence processing.

- **Expanded installer & Docker set** — bare-metal `install.sh` / `install_factory.sh` plus both a lab `docker-compose.yml` and the production `docker-compose.factory.yml`, and a light image under `docker/light/`.

1.1 What Changed from the Original Project

	Before (graduation prototype)	After (factory v1.1.16)
Running environment	Docker on laptop	Dedicated Linux server (bare-metal or Docker) at the factory
Traffic source	PCAP files / simulated traffic	Live SPAN/TAP port capture
OT detection	None	19 attack types across 6 OT protocols
Flood detection	Per-connection counters only	Sliding-window *and* single-connection volumetric detection
Authorized masters	Single Modbus list (reused for S7)	Separate Modbus and S7 master lists
Blocking safety	Blindly blocks any IP	OT-safe: PLCs/HMIs/SCADA servers are protected
Web interface	No login, unauthenticated API	Flask login with session management
Data persistence	Redis resets on restart	AOF + RDB persistence, survives reboots
Alert export	Slack and STIX only	Added Syslog UDP/TCP and HTTP webhook for SIEM
Config: direction	`analysis_direction: out` (IT only)	`all` (detects inbound OT attacks)
Sensitivity	`evidence_detection_threshold 0.25`	`0.15` for faster OT alerts

1.2 Key Files in the Factory Build

File	What It Does
`modules/ot_detection/ot_detection.py`	Main OT module — 19 attack types, 6 OT protocols, sliding-window *and* single-connection flood counters, OT-safe blocking, deduplication
`zeek-scripts/ot_protocols.zeek`	Zeek script — hooks Modbus PDU events, DNP3 headers, S7Comm connections, PROFINET DCP, PTP; emits structured notices
`modules/exporting_alerts/syslog_exporter.py`	Syslog (UDP/TCP, RFC-5424) and HTTP webhook export for SIEM integration
`webinterface/auth.py`	Flask login page, session guard, 8-hour sessions, credential config via environment variables
`config/ot_protected_devices.conf`	List of PLC/HMI/SCADA IPs — these are never auto-blocked
`config/modbus_authorized_masters.conf`	Authorized Modbus master IPs — others trigger `OT_UNAUTHORIZED_MODBUS_ACCESS`
`config/s7_authorized_masters.conf` NEW	Authorized S7Comm master IPs (TIA Portal, WinCC, SCADA with S7 drivers); falls back to the Modbus list if absent
`config/redis.conf`	Production Redis config: AOF persistence, memory limits, RDB snapshots

File	What It Does
`docker/docker-compose.factory.yml`	Production Docker Compose: Redis volume, healthchecks, restart policies, isolated `smartshield_net`, separate web container
`docker/docker-compose.yml` NEW	Lighter lab/pre-production compose for testing on a workstation
`install/install_factory.sh`	One-shot Ubuntu 22.04/24.04 bare-metal installer — Zeek 8, Python, Redis, zkg OT packages, systemd, credentials
`install/install.sh`, `install/requirements.txt`, `install/apt_dependencies.txt` NEW	Standard (non-factory) installer plus explicit pip and apt dependency manifests
`install/smartshield.service`	Systemd unit: restart-on-failure, resource limits, boot auto-start
`install/environment.template`	Environment-variable template for credentials and interface name

2. Factory Network Architecture

SmartShield is deployed at the IT/OT boundary following the ISA-95 / Purdue Model for industrial network segmentation. It monitors all traffic passively — OT devices never know it exists.

2.1 Purdue Model Network Zones

Zone	Name	Contains
Level 0–1	Field devices	PLCs, RTUs, VFDs, sensors, actuators. Communicate via Modbus, S7Comm, DNP3, PROFINET, CIP.
Level 2	Control	HMIs, engineering workstations, DCS systems. Run TIA Portal, SCADA client software.
Level 3	Operations	SCADA servers, historians (OSIsoft PI), OPC-UA servers, MES systems.
IT/OT Boundary	DMZ	**SmartShield server sits here.** SPAN/TAP receives all traffic crossing the boundary.
Level 4–5	IT / Enterprise	Business systems, ERP, internet. Standard IT security applies above this line.

2.2 SmartShield Server Placement

Physical placement rule

The SmartShield server MUST be connected via two separate interfaces:

- **eth0 (management):** SSH access, web UI on port 55000, syslog output. Connected to the IT/management VLAN.
- **eth1 (monitoring):** Connected to a SPAN port or TAP at the IT/OT boundary. Receives a mirror of all traffic. *This interface does NOT need an IP address.*

Never use your management interface for traffic capture — it will flood the interface and make SSH unusable.

2.3 Data Flow Through SmartShield

Stage	What Happens
SPAN/TAP port	Switch mirrors traffic to eth1. SmartShield sees a copy of every packet without disrupting OT communications.
Zeek	Parses raw packets into structured logs: `conn.log`, `modbus.log`, `dnp3.log`, `notice.log`, etc.
Redis pub/sub	Zeek events flow onto Redis channels. All SmartShield modules read from Redis, not directly from Zeek.
OT Detection module	Subscribes to `new_flow`, `new_modbus`, `new_s7comm`, `new_dnp3`, `new_notice`. Runs detection logic.
Evidence pipeline	Evidence objects flow through `evidence_handler.py` → `alert_handler.py` → `alerts.json`.
Blocking module	Subscribes to `new_blocking`. Checks the OT-safe list before adding iptables rules.
Export module	Subscribes to `export_evidence`. Sends to Slack, STIX, Syslog, or HTTP webhook.
Web interface	Flask app reads Redis directly. Shows profiles, alerts, evidence, blocked IPs in real time.

3. OT/ICS Attacks Detected

The `modules/ot_detection/ot_detection.py` module detects 19 attack categories. Every detection generates an Evidence object that flows through the normal SmartShield pipeline — visible in the web interface, written to `alerts.json`, and exported to any configured SIEM.

New: single-connection volumetric flood detection **v1.1.16**

In addition to per-connection sliding-window counters, every flow on an OT port is now also checked for high *volume in a single connection*. If one flow carries at least `connection_min_flood_packets` (default 200) and sustains at least `packet_rate_threshold` packets/second (default 50), a flood alert is raised at **HIGH** with 0.90 confidence. This closes the gap where a `hping3 --flood` style attack — logged by Zeek as one long-lived flow — would previously slip past the connection-count thresholds.

3.1 Modbus/TCP — Port 502

Evidence Type	Threat	Detection Logic
`OT_MODBUS_CONNECTION_FLOOD`	**HIGH**	Too many TCP connections/min to port 502 from one source (default threshold: 20/min), or one high-volume connection past the volumetric threshold
`OT_UNAUTHORIZED_MODBUS_ACCESS`	**HIGH**	Source IP not in `modbus_authorized_masters.conf` contacts a protected OT device on port 502
`OT_MODBUS_WRITE_COILS`	**CRITICAL**	Write Single/Multiple Coils or Registers (func codes 5, 6, 15, 16, 22, 23) from an unauthorized host
`OT_MODBUS_ILLEGAL_FUNCTION`	**MEDIUM**	Malformed function code (>127) or Modbus exception response from a device

3.2 Siemens S7Comm / ISO-TSAP — Port 102

S7 now uses its own authorized-master list v1.1.16

Unauthorized S7 read/write detection reads `config/s7_authorized_masters.conf` and no longer borrows the Modbus list. If that file is missing, it falls back to `modbus_authorized_masters.conf` so existing deployments keep working.

Evidence Type	Threat	Detection Logic
`OT_S7COMM_JOB_FLOOD`	HIGH	High-rate S7 job requests (TCP connections to port 102) within a sliding 60s window (threshold: 50), or a single high-volume session
`OT_S7COMM_UNAUTHORIZED_READ`	HIGH	S7Comm connection from a host not in the S7 authorized-master list targeting a protected OT device
`OT_S7COMM_UNAUTHORIZED_WRITE`	CRITICAL	S7 WriteVar command (ROSCTR=1, function 0x05) from an unauthorized source
`OT_S7COMM_PLC_STOP`	CRITICAL	S7 STOP / control-plane command (function 0x29 or 0x00, ROSCTR=1) — halts the PLC immediately

3.3 DNP3 — Port 20000

Evidence Type	Threat	Detection Logic
`OT_DNP3_FLOOD`	HIGH	High-rate DNP3 fragment/request flood (default threshold: 100 requests/60s), or a single high-volume session
`OT_DNP3_UNAUTHORIZED_FUNCTION`	HIGH	Function code > 33, which is reserved/dangerous in the DNP3 spec — potential exploitation attempt

3.4 Industrial Network Protocols

Evidence Type	Threat	Detection Logic
`OT_PROFINET_DCP_FLOOD`	MEDIUM	PROFINET DCP discovery flood (port 34964 UDP) — can overwhelm embedded PLCs with limited stack memory
`OT_CIP_MESSAGE_FLOOD`	HIGH	EtherNet/IP CIP explicit message flood (port 44818 TCP) — exhausts connection slots on Allen-Bradley controllers
`OT_NTP_TIME_SYNC_ATTACK`	HIGH	Possible NTP spoofing: > 5 NTP responses in 10s from the same source — desynchronizes precision OT clocks
`OT_PTP_SPOOFING`	HIGH	IEEE 1588 PTP event message (port 319 UDP) from a non-OT-known source — grandmaster clock spoofing

3.5 PLC Command and Process Attacks

Evidence Type	Threat	Detection Logic
`OT_PLC_MODE_SWITCH`	CRITICAL	Rapid RUN/STOP mode-switch commands to a PLC (threshold: 5 switches/60s) — can crash the PLC into a fault state

Evidence Type	Threat	Detection Logic
`OT_WATCHDOG_MANIPULATION`	CRITICAL	Watchdog timer disabled or manipulated (from Zeek `OT::WatchdogManipulation` notice) — safety systems bypassed
`OT_VALVE_CYCLING`	CRITICAL	Repeated open/close valve commands (`OT::ValveCycling` notice) — causes mechanical wear and process disruption
`OT_MOTOR_CYCLING`	CRITICAL	Rapid motor start/stop commands (`OT::MotorCycling` notice) — causes overheating and winding damage
`OT_PROTOCOL_ANOMALY`	MEDIUM	Generic OT protocol anomaly from Zeek OT notices — catch-all for patterns not covered by specific detectors

3.6 Confidence and Threat Levels

Every Evidence object has a confidence score (0.0–1.0) and a threat level. The `evidence_detection_threshold` in `smartshield.yaml` (set to 0.15 for the factory build) controls how much accumulated evidence is needed before an Alert is triggered. OT attacks use higher confidence scores (0.85–0.95) because protocol behavior is well-defined.

Threat Level	Evidence Types Assigned
CRITICAL (0.95)	`OT_S7COMM_PLC_STOP`, `OT_S7COMM_UNAUTHORIZED_WRITE`, `OT_MODBUS_WRITE_COILS`, `OT_PLC_MODE_SWITCH`, `OT_WATCHDOG_MANIPULATION`, `OT_VALVE_CYCLING`, `OT_MOTOR_CYCLING`
HIGH (0.80–0.90)	`OT_MODBUS_CONNECTION_FLOOD`, `OT_UNAUTHORIZED_MODBUS_ACCESS`, `OT_S7COMM_JOB_FLOOD`, `OT_S7COMM_UNAUTHORIZED_READ`, `OT_DNP3_FLOOD`, `OT_DNP3_UNAUTHORIZED_FUNCTION`, `OT_CIP_MESSAGE_FLOOD`, `OT_NTP_TIME_SYNC_ATTACK`, `OT_PTP_SPOOFING`
MEDIUM (0.70–0.80)	`OT_MODBUS_ILLEGAL_FUNCTION`, `OT_PROFINET_DCP_FLOOD`, `OT_PROTOCOL_ANOMALY`

4. OT-Safe Blocking

Why this matters

If SmartShield ever blocks a PLC's IP with iptables, that PLC stops receiving commands from the SCADA system and the production line halts. This is the single most dangerous operational risk of running an IPS in an OT environment.

The OT-safe blocking system prevents this. PLCs, HMIs, and SCADA servers listed in `config/ot_protected_devices.conf` are **NEVER** passed to iptables — no matter what evidence is detected against them.

4.1 How OT-Safe Blocking Works

The blocking pipeline is modified at three levels:

1. `modules/blocking/blocking.py` — `_is_ot_protected()` is called before every iptables rule. If the IP is protected, the rule is skipped and an `[OT-SAFE]` entry is written to `blocking.log`.

2. `smartshield_files/core/database/redis_db/alert_handler.py` — `mark_as_ot_protected()`, `is_ot_protected()`, and `get_ot_protected_ips()` persist the protected set in the Redis key `ot_protected_ips`.
3. `modules/ot_detection/ot_detection.py` — when evidence is set against a protected OT IP, the description includes `[OT-SAFE: blocking suppressed]` so analysts know immediately.

	Behavior
What is NOT blocked	The protected OT device's own IP (e.g. the PLC at 192.168.10.10).
What IS still blocked	External IT-side hosts attacking OT devices. If 10.5.1.88 floods PLC 192.168.10.10, attacker 10.5.1.88 is blocked normally — only the PLC is protected.
What always happens	Evidence, alerts, and SIEM exports are generated regardless of blocking status.

4.2 Configuring OT Protected Devices

Edit `config/ot_protected_devices.conf`. Add one IP address or CIDR range per line. Lines starting with `#` are comments.

```
# config/ot_protected_devices.conf
# Level 0-1: PLCs, RTUs, VFDs (field devices)
192.168.10.10   # PLC-01 Siemens S7-1200 (Assembly Line A)
192.168.10.11   # PLC-02 Siemens S7-1500 (Assembly Line B)
192.168.10.12   # PLC-03 Allen-Bradley (Conveyor system)
192.168.10.20   # RTU-01 Remote Terminal Unit (substation)
192.168.10.30   # VFD-01 Variable Frequency Drive
# Level 2: HMIs and Engineering Workstations
192.168.20.10   # HMI-01 SCADA operator workstation
192.168.20.50   # EWS-01 Engineering workstation (TIA Portal)
# Level 3: SCADA Servers and Historians
192.168.30.10   # SCADA-Server-01
192.168.30.20   # Historian-01 OSIsoft PI
192.168.30.30   # OPC-UA Server
# CIDR ranges are also supported:
# 192.168.10.0/24   # Protect entire field-device subnet
```

4.3 Configuring Authorized Masters (Modbus and S7)

List every IP that is legitimately allowed to send read/write commands. Any unlisted host contacting a protected OT device triggers the matching unauthorized-access evidence.

```
# config/modbus_authorized_masters.conf  — one exact IP per line (CIDR not supported)
192.168.20.10   # HMI-01 (authorized to read/write PLCs)
192.168.20.50   # EWS-01 (engineering workstation, TIA Portal)
192.168.30.10   # SCADA-Server-01 (authorized polling)

# config/s7_authorized_masters.conf  — NEW: dedicated S7 list
192.168.20.50   # TIA Portal engineering station
192.168.20.11   # WinCC HMI
192.168.30.10   # SCADA server with S7 driver
```

Leave both files empty initially

If `ot_protected_devices.conf` is empty, OT-safe blocking is still active but no IPs are protected.

If an authorized-masters file is empty, unauthorized-access detection for that protocol is disabled (flood detection still works). If `s7_authorized_masters.conf` does not exist at all, the Modbus list is used for S7 as well.

Populate these files before enabling blocking in production.

5. Installation

5.1 Server Requirements

Requirement	Specification
Operating system	Ubuntu 22.04 LTS or 24.04 LTS (bare-metal or VM). Debian 12 also supported.
CPU	4+ cores recommended. Zeek is multi-threaded and can use 2–3 cores under load.
RAM	8 GB minimum. 16 GB recommended for busy factory networks (> 1 Gbps sustained).
Disk	50 GB minimum for OS, SmartShield, and Zeek logs. 200 GB recommended if storing rotated logs.
Network interfaces	Minimum 2: one management (eth0), one monitoring (eth1) connected to SPAN/TAP.
Monitoring interface	Does not need an IP address. Any speed that matches the SPAN output.
Python	Python 3.10+ (included in Ubuntu 22.04/24.04).
Zeek	Zeek 8.0 — installed automatically by `install_factory.sh`.
Redis	Redis 7/8 — installed and configured automatically. Upstream pins Redis ≥ 8.2.2 (CVE-2025-49844 patch) for the bundled images.
Root access	Required for installation, iptables, and packet capture.

5.2 Pre-Installation: Switch SPAN Port Setup

Before installing, configure your managed switch to mirror the IT/OT boundary port to the server's monitoring interface. The exact steps depend on the switch vendor.

Switch Vendor	Source Command	Destination / Notes
Cisco IOS / Catalyst	`monitor session 1 source interface Gi1/0/1`	`monitor session 1 destination interface Gi1/0/24`
Juniper EX	`set ... analyzer SPAN input ingress interface ge-0/0/1.0`	`set analyzer SPAN output interface ge-0/0/24.0`
HP / Aruba ProCurve	`mirror-session 1 port A1`	`mirror-port A24` (or web UI: Diagnostics → Mirror)
Siemens SCALANCE	Port Mirroring under Diagnostics menu	Source: IT/OT boundary port, Destination: SmartShield monitoring port
Generic / other	Look for: Port Mirroring, SPAN, Port Monitor, Traffic Mirror	Source = IT/OT boundary uplink, Destination = SmartShield eth1

Verify SPAN is working before installing

```
sudo ip link set eth1 up
sudo tcpdump -i eth1 -n -c 50
```

You should see traffic from your OT devices. If the interface is silent, the SPAN port is not configured correctly.

5.3 Option A: Bare-Metal Install (Recommended for Production)

The included installer handles everything on a fresh Ubuntu server. Run it as root.

```
# Step 1 — Copy the project to the factory server
scp -r SmartShield-Factory/ admin@<server-ip>:/opt/

# Step 2 — SSH into the server
ssh admin@<server-ip>

# Step 3 — Run the installer as root
cd /opt/SmartShield-Factory
sudo chmod +x install/install_factory.sh
sudo ./install/install_factory.sh

# The script will:
#  - Install Zeek 8, Python 3, Redis, iptables-persistent
#  - Install Zeek OT packages via zkg (ICS-Passive-Discovery)
#  - Ask for the monitoring interface name (e.g. eth1)
#  - Ask for web UI username and password (min 12 chars)
#  - Copy config to /etc/smartshield/
#  - Create /var/lib/smartshield/ for persistent Redis data
#  - Install and enable the smartshield systemd service
#  - Optionally start SmartShield immediately
```

After installation — edit the OT config files **before starting**:

```
sudo nano /opt/SmartShield-Factory/config/ot_protected_devices.conf
sudo nano /opt/SmartShield-Factory/config/modbus_authorized_masters.conf
sudo nano /opt/SmartShield-Factory/config/s7_authorized_masters.conf   # NEW

sudo systemctl start smartshield
sudo systemctl status smartshield
```

5.4 Option B: Docker (Lab / Pre-Production Testing)

Use Docker to test before deploying bare-metal, or in a virtualized environment. The factory build ships two compose files: `docker-compose.yml` for a quick lab run and `docker-compose.factory.yml` for the hardened production stack.

```
# Step 1 — Create the .env file
cat > docker/.env << 'ENVEOF'
SMARTSHIELD_INTERFACE=eth1
SMARTSHIELD_WEB_USER=admin
SMARTSHIELD_WEB_PASSWORD=YourStrongFactoryPassword!
SMARTSHIELD_SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))")
ENVEOF

# Step 2 — Create persistent storage directories on the host
sudo mkdir -p /var/lib/smartshield/{redis,output}

# Step 3 — Build the image
docker build -f docker/Dockerfile -t smartshield_gp:latest .

# Step 4 — Start the production stack
docker compose -f docker/docker-compose.factory.yml up -d

# Step 5 — Check status
docker compose -f docker/docker-compose.factory.yml ps
docker logs smartshield -f
```

Docker vs Bare-Metal

- The three containers (`smartshield` , `smartshield-redis` , `smartshield-web`) run on an isolated `smartshield_net` bridge network. Redis is reachable by the other containers by name but is not published to the host.
- iptables blocking requires `network_mode: host` , which reduces container isolation.
- Live interface capture with `NET_RAW` needs the host interface UP before starting.
- Redis volumes must be bind-mounted to a host path for true persistence.

Bare-metal is recommended for production deployments where stability is critical.

6. Configuration Reference

6.1 Main Configuration: `config/smartshield.yaml`

```
parameters:
  analysis_direction: all          # was: out — detect inbound attacks FROM IT TO OT
  deletePrevdb: false              # was: true — keep evidence across reboots
  rotation: true
  rotation_period: 1day
  keep_rotated_files_for: 7 day

detection:
  evidence_detection_threshold: 0.15   # was: 0.25 — alert faster on OT attacks

ot_detection:
  ot_devices_file:    config/ot_protected_devices.conf
  modbus_masters_file: config/modbus_authorized_masters.conf
  s7_masters_file:    config/s7_authorized_masters.conf   # NEW
  modbus_flood_threshold: 20       # connections/min before alerting
  s7_job_flood_threshold: 50       # S7 connections/min
  dnp3_flood_threshold:  100       # DNP3 requests/min
  profinet_dcp_threshold: 50       # PROFINET DCP requests/min
  cip_flood_threshold:   100       # CIP messages/min
  flood_window_seconds:   60       # sliding-window duration
  connection_min_flood_packets: 200   # NEW: single-conn volumetric floor
  packet_rate_threshold:  50          # NEW: single-conn packets/second floor
  plc_mode_switch_threshold: 5     # RUN/STOP switches in 60s

web_interface:
  port: 55000
```

Parameter	Purpose
`s7_masters_file` NEW	Path to the dedicated S7 authorized-master list. Falls back to the Modbus list if the file is missing.
`connection_min_flood_packets` NEW	Minimum packet count in a single OT-port flow before it can be flagged as a volumetric flood.
`packet_rate_threshold` NEW	Minimum packets/second a single flow must sustain to be flagged. Both conditions must hold.

6.2 SIEM / Syslog Export Configuration

```
exporting_alerts:
  export_to: [syslog]              # Options: slack, stix, syslog, webhook
  # Syslog target — Splunk, QRadar, Wazuh, Graylog, Elastic SIEM
  syslog_server: 192.168.1.100
  syslog_port: 514                 # 514 for UDP, 601 for TCP syslog
  syslog_protocol: udp             # "udp" or "tcp"
  syslog_facility: local0
  syslog_app_name: smartshield
  # HTTP Webhook — Splunk HEC, Grafana, PagerDuty, custom endpoint
  webhook_url: http://siem-server:8088/services/collector
  webhook_auth_token: your_splunk_hec_token
  webhook_timeout: 5
```

The syslog message format is structured for easy SIEM parsing:

```
smartshield: SmartShield ALERT | type=OT_MODBUS_WRITE_COILS |
 threat=CRITICAL | confidence=0.95 | attacker=10.5.1.88 |
 ts=2024/03/15 14:32:01.000000+0000 |
 desc=Unauthorized Modbus write from 10.5.1.88 to OT device 192.168.10.10...
```

6.3 Web Interface Credentials

```
# Bare-metal: edit /etc/smartshield/environment
SMARTSHIELD_WEB_USER=factoryadmin
SMARTSHIELD_WEB_PASSWORD=MyFactory_Secure_Pass2024!

# Generate a cryptographic secret key (run once):
python3 -c "import secrets; print(secrets.token_hex(32))"
SMARTSHIELD_SECRET_KEY=<paste output here>

# Docker: set in docker/.env
# Restart after changes: sudo systemctl restart smartshield
```

6.4 Redis Persistence Configuration

The factory `config/redis.conf` enables both AOF (append-only file) and RDB snapshots:

```
appendonly yes
appendfilename "smartshield.aof"
appendfsync everysec             # flush to disk every second (best balance)
save 900 1                       # snapshot after 900s if ≥1 key changed
save 300 10
save 60 10000
maxmemory 512mb
maxmemory-policy allkeys-lru
```

Redis binding in the factory build v1.1.16

The factory `redis.conf` uses `bind 0.0.0.0` so the SmartShield and web containers can reach Redis over the isolated `smartshield_net` Docker network. Redis is **not** published to the host or the wider network — isolation comes from the Docker network and the host firewall, not from a localhost bind. On bare-metal single-host installs you may set `bind 127.0.0.1` instead. Data persists in `/var/lib/smartshield/redis/` across reboots and container restarts.

7. Running and Managing SmartShield

7.1 Starting for the First Time — Recommended Sequence

1. **Populate OT config files.** Add all PLC/HMI/SCADA IPs to `ot_protected_devices.conf` and all authorized masters to `modbus_authorized_masters.conf` and `s7_authorized_masters.conf`. Mandatory before enabling blocking.
2. **Start in monitor-only mode first.** Do not enable `--blocking` on the first run. Let SmartShield observe for 1–2 weeks to establish a baseline. Watch for false positives.
3. **Verify SPAN capture.** Run `sudo tcpdump -i eth1 -n port 502 or port 102 -c 20` to confirm OT traffic is arriving.
4. **Check the web interface.** Open `http://<server-ip>:55000` and log in. Profiles should appear within a few minutes of capture starting.
5. **Tune thresholds.** Review alerts after 1 week. Adjust `modbus_flood_threshold` and `s7_job_flood_threshold` based on your factory's normal polling rates.
6. **Enable blocking.** Add `--blocking` to the start command, then restart. Verify no OT device IPs appear in the rules: `sudo iptables -L smartshieldBlocking -n`.
7. **Configure SIEM export.** Add `syslog` to `export_to`, set `syslog_server`, then restart. Verify alerts arrive in your SIEM.

7.2 Systemd Commands (Bare-Metal)

Command	Description
`sudo systemctl status smartshield`	Check if running. Shows last log lines.
`sudo systemctl start smartshield`	Start the service.
`sudo systemctl stop smartshield`	Stop the service. Redis data is preserved.
`sudo systemctl restart smartshield`	Restart after config changes.
`sudo journalctl -u smartshield -f`	Live log stream (Ctrl+C to stop).
`sudo journalctl -u smartshield -p err`	Show only errors.

7.3 Docker Commands

Command	Description
`docker compose -f docker/docker-compose.factory.yml up -d`	Start all services in background.
`docker compose -f docker/docker-compose.factory.yml down`	Stop all services. Volumes (Redis data) preserved.
`docker logs smartshield -f --tail 100`	Follow SmartShield container logs.
`docker stats`	Live CPU and memory usage for all containers.
`docker exec -it smartshield-redis redis-cli ping`	Test Redis connectivity.

Command	Description
`docker exec -it smartshield-redis redis-cli info persistence`	Check AOF/RDB persistence status.

7.4 Useful Diagnostic Commands

```
# Network verification
sudo tcpdump -i eth1 -n port 502 or port 102 or port 20000 -c 30
sudo timeout 10 tcpdump -i eth1 -n port 502 | wc -l

# iptables blocking rules
sudo iptables -L smartshieldBlocking -v -n
sudo iptables -L smartshieldBlocking -n | grep 192.168.10     # OT subnet must be absent

# Redis status
redis-cli INFO persistence
redis-cli SMEMBERS ot_protected_ips
redis-cli KEYS "evidence_*" | wc -l
redis-cli KEYS "alert_*" | wc -l

# SmartShield output
tail -f /var/lib/smartshield/output/alerts.json
cat /var/lib/smartshield/output/blocking.log

# Zeek status
zeek --version
ls /opt/zeek/logs/current/
```

8. Web Interface

8.1 Accessing the Web Interface

Setting	Value
URL	`http://<server-management-ip>:55000`
Username	Set in `SMARTSHIELD_WEB_USER` (default: `admin`)
Password	Set in `SMARTSHIELD_WEB_PASSWORD` (default shows a warning — must be changed)
Session	8-hour sessions. Expires automatically. All routes require login.
Port change	Edit `web_interface.port` in `smartshield.yaml`, then restart.

Restrict access to port 55000

```
sudo ufw allow from 192.168.20.0/24 to any port 55000
sudo ufw deny 55000
```

Never expose port 55000 to the internet. Use a VPN if remote access is needed.

8.2 Web Interface Pages

Page	Contents
Overview (`/`)	Summary of profiles, alert count, analysis start time, input type.
Analysis (`/analysis`)	Per-IP profiles, time windows, evidence list, flow details, blocking status.
Timeline	Chronological event timeline for each IP — shows attack progression.
Documentation (`/documentation`)	Built-in SmartShield documentation reference.

8.3 Authentication Implementation

The `webinterface/auth.py` module adds login protection using Flask sessions:

- **No external dependencies:** uses Flask's built-in session system with a secret key. No database required for auth.
- **Before-request guard:** every request (except `/login`, `/logout`, `/static`) is intercepted and redirected to login if no valid session exists.
- **Credential source:** usernames and passwords come from environment variables only — not from config files that might be committed to git.
- **Session lifetime:** 8 hours, via `app.permanent_session_lifetime`.
- **Brute-force protection:** failed logins add a 0.5-second delay to slow automated attacks.
- **Secret key:** must be a random 32-byte hex string. If `SMARTSHIELD_SECRET_KEY` is unset, a random key is generated each restart (invalidating existing sessions).

9. OT Detection Module — Technical Details

9.1 Module Architecture

The OT module follows the same `IModule` interface as all other SmartShield modules and is auto-discovered because the file name matches the directory name (`ot_detection/ot_detection.py`). It subscribes to five Redis pub/sub channels:

Channel	Source and Contents
`new_flow`	Every Zeek `conn.log` entry — used for sliding-window and single-connection flood detection on OT ports (502, 102, 20000, 34964, 44818, 123, 319)
`new_notice`	Zeek `notice.log` — catches `PLCModeSwitch`, `WatchdogManipulation`, `ValveCycling`, `MotorCycling` from `ot_protocols.zeek`
`new_modbus`	Per-PDU Modbus events — function code, exception flag, coil values
`new_s7comm`	Per-PDU S7Comm events — ROSCTR, function code, read/write operations
`new_dnp3`	Per-PDU DNP3 events — function code, is_unsolicited, data objects

Channels that are not registered by the active Zeek build are skipped gracefully — the module keeps any channel that subscribed successfully and falls back to `conn.log` -based flood detection for the rest.

9.2 Flood Detection — Two Complementary Methods

(a) Sliding-window counters count discrete connections per source over a moving window, avoiding boundary-crossing evasion:

```
now = time.time()
self._modbus_conns[saddr].append(now)
self._modbus_conns[saddr] = [t for t in self._modbus_conns[saddr]
                             if now - t <= self._flood_window]   # default 60s
if len(self._modbus_conns[saddr]) >= self._modbus_flood_threshold:   # default 20
    self._set_evidence(EvidenceType.OT_MODBUS_CONNECTION_FLOOD, ...)
```

(b) Single-connection volumetric detection NEW — a flood riding one long-lived flow is logged by Zeek as a single `conn.log` entry, so the counters above never trip. The module reads the packet counts and duration Zeek already records:

```
flow_pkts = int(flow.get("spkts", 0)) + int(flow.get("dpkts", 0))
dur  = float(flow.get("dur", 0) or 0)
rate = (flow_pkts / dur) if dur > 0 else float("inf")
if (flow_pkts >= self._min_flood_packets        # default 200
        and rate >= self._packet_rate_threshold):  # default 50 pkts/s
    self._set_evidence(ev_type, threat_level=ThreatLevel.HIGH, confidence=0.9, ...)
```

9.3 Alert Deduplication

The module keeps a `self._alerted` set of alert keys (e.g. `"modbus_flood:<srcip>:<twid>"`, `"vol_flood:<src>:<dst>:<port>:<twid>"`) to prevent the same attack from generating hundreds of duplicate evidence objects. The set is per-instance and resets when the module restarts. Within a time window, each unique (attack_type, source, dest) combination generates exactly one Evidence object.

9.4 Zeek OT Protocol Script

`zeek-scripts/ot_protocols.zeek` is loaded via `__load__.zeek`. It hooks Zeek's built-in Modbus and DNP3 policies, tracks S7Comm at the TCP level (Zeek has no full S7 parser), detects PROFINET DCP floods on port 34964 UDP, and watches PTP on port 319 UDP. It defines custom notices: `PLCModeSwitch`, `WatchdogManipulation`, `ValveCycling`, `MotorCycling`, `ModbusWriteCoil`, `ModbusIllegalFunction`, `S7CommPLCStop`, `DNP3UnauthorizedFunc`, `OTProtocolAnomaly`.

If Zeek's OT policy bundle is unavailable (older installations), `ot_protocols.zeek` partially fails but SmartShield falls back gracefully to `conn.log`-based detection, which still handles flood attacks on OT ports.

9.5 Evidence Object Structure

```
Evidence(
  evidence_type = EvidenceType.OT_MODBUS_WRITE_COILS,
  description   = "Unauthorized Modbus write from 10.5.1.88 to OT device",
  attacker = Attacker(direction=Direction.SRC, ioc_type=IoCType.IP, value="10.5.1.88"),
  victim   = Victim(direction=Direction.DST, ioc_type=IoCType.IP, value="192.168.10.10"),
  threat_level = ThreatLevel.CRITICAL,
  profile      = ProfileID(ip="10.5.1.88"),
  timewindow   = TimeWindow(number=3),
  uid          = ["CYbGfv3fLZzYMVqj12"],
  timestamp    = "2024/03/15 14:32:01.000000+0000",
  proto = Proto.TCP, dst_port = 502, confidence = 0.95,
)
```

10. Tuning and Optimization

10.1 Baseline Period

OT networks have highly predictable, cyclic traffic — PLCs poll sensors at fixed intervals, HMIs refresh at fixed rates. This predictability aids detection but requires careful tuning to avoid false positives.

Recommended tuning approach

- **Week 1–2:** Monitor only (no blocking). Collect baseline data; note the normal Modbus polling rate from your SCADA server.
- **Week 3:** Lower flood thresholds toward observed normal + 30% headroom. Watch for false positives.
- **Week 4:** Enable blocking with the OT-safe list fully populated. Start high, tighten over time.
- **Ongoing:** Review `blocking.log` weekly. Any `[OT-SAFE]` entries indicate attacks on OT devices needing manual investigation.

10.2 Flood Threshold Tuning Guide

Parameter	How to Tune	Example
`modbus_flood_threshold`	SCADA polling rate × 2	5 PLCs polled every 5s → ~60 conn/min → set 120
`s7_job_flood_threshold`	TIA Portal polling rate × 3	Workstations poll ~1 req/s; attacks 50+/s. Keep at 50.
`dnp3_flood_threshold`	RTU polling rate × 3	RTUs send 1–5 frames/s; attacks 100+/s. Start at 100.
`profinet_dcp_threshold`	Near 0 in normal operation	DCP discovery only fires at device startup. Sustained > 10 is suspicious.
`connection_min_flood_packets` NEW	Above the largest legitimate single OT session	200 packets is a safe floor for most polling sessions; raise if a historian pulls large blocks.
`packet_rate_threshold` NEW	~10× normal per-flow packet rate	Normal polling is a few pkts/s; 50 pkts/s in one flow is flood territory.

Parameter	How to Tune	Example
`plc_mode_switch_threshold`	Set to 2 if one SCADA can switch	A legitimate switch is 1 per maintenance window. 5 in 60s is always an attack.

10.3 Evidence Detection Threshold

Threshold	Effect
0.08	Very sensitive. Use during initial testing; expect false positives. Good for validating detections.
0.15	**Current factory setting.** Balanced for OT environments where attacks are sudden and high-confidence.
0.25	Original default. Suitable for noisy IT networks. Too slow for OT attack response.
0.43	Insensitive. Alerts only after many corroborating evidence objects. Risk of missing fast attacks.

10.4 Performance Considerations

- **Zeek CPU:** the heaviest process. At 1 Gbps sustained it uses 2–3 cores. Scale server cores accordingly.
- **Redis memory:** default limit 512 MB. Each IP profile uses ~50 KB → ~10,000 concurrent profiles.
- **OT detection overhead:** negligible CPU. Sliding-window cleanup is O(n), bounded by the threshold; volumetric detection reuses counters Zeek already computed, so it adds no extra parsing.
- **Log rotation:** `rotation: true` / `rotation_period: 1day`; logs deleted after `keep_rotated_files_for: 7 day`.
- **Bloom-filter speedups:** v1.1.16 uses bloom filters in evidence/whitelist handling, reducing per-flow lookup cost on busy networks.

11. Troubleshooting

11.1 Common Problems and Solutions

Problem	First Check	Solution / Cause
OT module not loading	`journalctl -u smartshield | grep OT`	Missing `__init__.py` → `touch modules/ot_detection/__init__.py` and restart.
No Modbus alerts despite flood traffic	`tcpdump -i eth1 port 502`	SPAN not configured; threshold too high; or the attacker IP is in `ot_protected_devices.conf`.
hping3 SYN flood not alerting	`redis-cli KEYS "alert_*"`	If on a very short burst, raise sensitivity; the volumetric detector needs `connection_min_flood_packets` packets in one flow. Lower it for a faster demo.
S7 unauthorized false positives	Check `s7_authorized_masters.conf`	Add the legitimate S7 client (TIA Portal / WinCC / SCADA) IP, or leave the file empty to disable the check.

Problem	First Check	Solution / Cause
Web UI shows no alerts after 1 hour	`redis-cli KEYS "alert_*"`	Threshold too high, or alerts only appear after 1 time-window. Lower to 0.08 for testing.
Redis data lost after restart	`redis-cli INFO persistence`	Look for `aof_enabled:1`. Verify `appendonly yes` and the path is writable.
Cannot log into web interface	`curl -I http://localhost:55000/login`	`SMARTSHIELD_WEB_USER/PASSWORD` not set. Check the environment file and restart.
Syslog not reaching SIEM	`echo "test" | nc -u <siem-ip> 514`	Check firewall; verify `syslog_server` and `syslog_port`.
Factory line halted — PLC blocked	URGENT — see 11.2	Remove the iptables rule, add the PLC IP to `ot_protected_devices.conf`, restart.

11.2 Emergency Procedure — PLC Blocked

If SmartShield blocks an OT device

```
# 1. Immediately remove the iptables rule
sudo iptables -D smartshieldBlocking -s <plc-ip> -j DROP
sudo iptables -D smartshieldBlocking -d <plc-ip> -j DROP

# 2. Add the PLC IP to the protected list
echo "192.168.10.10 # PLC-01 EMERGENCY ADDED" >> config/ot_protected_devices.conf

# 3. Restart to reload the protected list
sudo systemctl restart smartshield

# 4. Verify the IP no longer appears in blocking rules
sudo iptables -L smartshieldBlocking -n | grep 192.168.10.10

# 5. Investigate the original alert to determine if there was a real attack.
```

11.3 Log File Locations

Log File	Location and Contents
`alerts.json`	`/var/lib/smartshield/output/alerts.json` — all generated alerts in JSON
`blocking.log`	`/var/lib/smartshield/output/blocking.log` — blocking/unblocking events with `[OT-SAFE]` tags
`smartshield.log`	`/var/lib/smartshield/output/smartshield.log` — main application log
`errors.log`	`/var/lib/smartshield/output/errors.log` — error and exception log
Zeek `conn.log`	`/var/lib/smartshield/output/zeek/conn.log` — all parsed connections
Redis AOF	`/var/lib/smartshield/redis/smartshield.aof` — persistent Redis command log
systemd journal	`journalctl -u smartshield` — full service log

12. Security Hardening Checklist

Network Hardening

- **SPAN port:** monitoring interface (eth1) has no IP. `ip addr show eth1` shows no `inet` line.
- **Firewall:** port 55000 only accessible from the management VLAN. `sudo ufw status numbered`.
- **Management interface:** eth0 has only 22/SSH and 55000/Web open. No services on eth1.
- **Redis exposure:** Redis is reachable only on the isolated `smartshield_net` Docker network (or `127.0.0.1` on bare-metal). It is not published to the host or LAN. Confirm with `docker port smartshield-redis` (should be empty or localhost only).

Authentication Hardening

- **Default password changed:** `SMARTSHIELD_WEB_PASSWORD` is not the default. No warning banner in the web UI.
- **Secret key set:** `SMARTSHIELD_SECRET_KEY` is a 32-byte random hex string.
- **Environment file permissions:** `chmod 600 /etc/smartshield/environment`.
- **No credentials in git:** `.env` and environment files are in `.gitignore`.

OT Safety Verification

- **OT device list populated:** `ot_protected_devices.conf` has all PLCs, HMIs, SCADA servers.
- **Protected IPs in Redis:** `redis-cli SMEMBERS ot_protected_ips` lists all configured OT IPs.
- **No OT IPs blocked:** `sudo iptables -L smartshieldBlocking -n` shows no PLC/HMI/SCADA IPs.
- **Master lists populated:** both `modbus_authorized_masters.conf` and `s7_authorized_masters.conf` reviewed.
- **blocking.log reviewed:** no unexpected `[OT-SAFE]` entries — each means an attack hit a protected device.

Data Persistence & Monitoring

- **AOF enabled:** `redis-cli INFO persistence` shows `aof_enabled:1`.
- **Reboot test:** `sudo reboot`, then verify auto-start and that evidence history is preserved.
- **Syslog tested:** trigger a test alert; verify it reaches the SIEM within 30 seconds.
- **Systemd watchdog:** restart policy is on-failure. `sudo kill -9 $(pgrep -f smartshield.py)` — service restarts within ~10 seconds.
- **Disk monitor:** alert if `/var/lib/smartshield` exceeds 80% full.

13. Quick Reference

13.1 All OT Evidence Types (19)

Evidence Type	Threat	Description
`OT_MODBUS_CONNECTION_FLOOD`	HIGH	Port 502 TCP flood (connection-count or single-connection volumetric)
`OT_MODBUS_WRITE_COILS`	CRITICAL	Unauthorized Modbus write coil/register
`OT_MODBUS_ILLEGAL_FUNCTION`	MEDIUM	Malformed / exception Modbus function code
`OT_UNAUTHORIZED_MODBUS_ACCESS`	HIGH	Modbus from a non-authorized master IP
`OT_S7COMM_JOB_FLOOD`	HIGH	S7 port 102 connection flood
`OT_S7COMM_UNAUTHORIZED_READ`	HIGH	S7 connection from non-authorized source
`OT_S7COMM_UNAUTHORIZED_WRITE`	CRITICAL	S7 WriteVar from unauthorized host
`OT_S7COMM_PLC_STOP`	CRITICAL	S7 STOP command sent to PLC
`OT_DNP3_FLOOD`	HIGH	DNP3 request flood (port 20000)
`OT_DNP3_UNAUTHORIZED_FUNCTION`	HIGH	DNP3 function code > 33
`OT_PROFINET_DCP_FLOOD`	MEDIUM	PROFINET DCP discovery flood (port 34964)
`OT_CIP_MESSAGE_FLOOD`	HIGH	EtherNet/IP CIP explicit message flood (port 44818)
`OT_NTP_TIME_SYNC_ATTACK`	HIGH	NTP spoofing — > 5 responses/10s
`OT_PTP_SPOOFING`	HIGH	PTP grandmaster spoofing (port 319 UDP)
`OT_PLC_MODE_SWITCH`	CRITICAL	Rapid PLC RUN/STOP mode switching
`OT_WATCHDOG_MANIPULATION`	CRITICAL	Watchdog timer disabled or manipulated
`OT_VALVE_CYCLING`	CRITICAL	Rapid valve open/close commands
`OT_MOTOR_CYCLING`	CRITICAL	Rapid motor start/stop commands
`OT_PROTOCOL_ANOMALY`	MEDIUM	Generic OT protocol anomaly (Zeek notice)

13.2 Key File Paths

Item	Path
Main config	`/opt/SmartShield-Factory/config/smartshield.yaml`
OT protected devices	`config/ot_protected_devices.conf`
Authorized Modbus masters	`config/modbus_authorized_masters.conf`
Authorized S7 masters NEW	`config/s7_authorized_masters.conf`
Redis config	`config/redis.conf`
Service environment file	`/etc/smartshield/environment`
Systemd service	`/etc/systemd/system/smartshield.service`
Redis data	`/var/lib/smartshield/redis/`

Item	Path
Alerts output	`/var/lib/smartshield/output/alerts.json`
Blocking log	`/var/lib/smartshield/output/blocking.log`

13.3 Port Reference

Port	Protocol	OT Devices
502/TCP	Modbus/TCP	PLCs, remote I/O modules, Modbus gateways
102/TCP	S7Comm (ISO-TSAP)	Siemens S7-300/400/1200/1500 PLCs
20000/TCP+UDP	DNP3	RTUs, substations, water/power utilities
4840/TCP	OPC-UA	SCADA servers, historians, modern PLC gateways
34964/UDP	PROFINET DCP	Siemens PROFINET devices, distributed I/O
44818/TCP	EtherNet/IP CIP	Allen-Bradley (Rockwell) ControlLogix/CompactLogix
123/UDP	NTP	Time sync for PLCs and SCADA servers
319/UDP	PTP (IEEE 1588)	Precision time for motion control and substations
55000/TCP	SmartShield Web UI	Management interface — restrict to management VLAN
6379/TCP	Redis	Internal only — Docker network / localhost binding